

AS02 Memo: A 270M-Parameter LoRA Classifier Beats Its Own Base Model at Reading Patient Medication Reports, but the Rating-Derived “Neutral” Label Is the Weak Link

Scenario 01 — Personalized Medication Decision Support

Yara Yao

Course: 95-864 AI Model Development, Heinz College, Carnegie Mellon University

September 28, 2026

Abstract

This memo reports on finetuning a small open-weight language model (Google `gemma-3-270m-it`, 270M parameters) with LoRA inside the course’s LLMBox application, so that it reads a patient’s own report about a medication and returns one of three experience classes (`POSITIVE`, `NEUTRAL`, `NEGATIVE`) for clinicians and pharmacists. We transformed the Drugs.com review corpus from Assignment 1 into 3,000 balanced, de-duplicated, privacy-filtered prompt/label pairs, ran three controlled finetuning experiments (LLMBox defaults as a baseline, an updated configuration, and an attention-only LoRA variant), and evaluated the models on 100 held-out reports, 100 reports about drugs never seen in training, 24 hand-written edge cases, and with a Pydantic-validated LLM judge. Three findings stand out. (1) Finetuning works even at this scale: accuracy on held-out reports rose from 0.46 (base model, which never predicts `NEUTRAL`) to 0.62 (macro-F1 $0.37 \rightarrow 0.63$) and dangerous polarity flips (positive read as negative or vice versa) fell from 18 to 0 per 100 reports. (2) The single most valuable configuration change was economic, not statistical: raising the micro-batch from 2 to 8 lifted GPU utilisation from 27% to 44%, cut training time by $5.5\times$ and energy by 73%, while the higher learning rate also gave the lowest validation loss (0.207 vs. 0.226). (3) The remaining errors are concentrated in `NEUTRAL`: a stronger independent judge model (Phi-4-mini) agreed with the rating-derived gold label on only 57% of reports and read 27 of 36 “neutral” reports as negative, while the human rater and the judge agreed with each other on 82% of ratings (kappa 0.64), so the label definition, not the model, is now the bottleneck. We recommend the organisation ship the finetuned model only as a triage aid with a confidence threshold, re-label the neutral band with clinicians before further training, and treat batch size and target-module choice as cost levers to be measured, not defaults to be inherited. More broadly: balance and de-duplicate before splitting, keep an unseen-entity test set, and never report a single accuracy number without the class-specific recall that matters for safety.

1 Introduction

Organisational task. Our organisation is a medical-AI start-up piloting with a network of clinics that treat chronic conditions (Assignment 1, Scenario 01). Clinicians and pharmacists want a “patient-voice” summary of how a drug is tolerated by people like their patient. The atomic task is: given the drug, the condition it was prescribed for and a patient’s free-text report, output the

reported experience class. Aggregated over thousands of reports this ranks candidate drugs by tolerability and surfaces common complaints. The tool is decision support only: it never gives dosing advice and never replaces clinical judgement.

Why it matters, and why with AI. Patient-reported outcomes are the only large-scale evidence about tolerability in everyday use, but they are unstructured and too voluminous to read. A model that reliably labels them turns free text into a ranking a pharmacist can act on. Doing it with a *small, local* model matters for this organisation specifically: reports may contain protected health information, so sending them to a commercial API is a governance problem; and a 270M-parameter model runs on a clinic laptop.

Research questions.

- RQ1. Can a 270M instruction-tuned model be shifted by LoRA finetuning from “chatty assistant” to a reliable three-class labeller on organisational data?
- RQ2. Which training-configuration choices move the LLMBox economic metrics (time, memory, energy) and which move the performance metrics (loss, perplexity), and are the two in tension?
- RQ3. Where does the finetuned model fail, and are those failures a property of the model or of the labels?

Hypotheses and what informed them. From Assignment 1 we knew that generic embeddings separate reviews by *condition* but not by *experience* (all methods at or below the majority-class baseline), so we expected the base model to be poor and supervised finetuning to be necessary. From the LoRA paper [7] and the PyTorch/Hugging Face consumer-hardware finetuning post [8] we expected that adapting under 1% of the weights would be enough for a narrow classification behaviour, and that batch size would dominate wall-clock cost on a GPU. From the AS01 datasheet we expected the NEUTRAL class (ratings 5–6) to be the hardest because it is a band on an ordinal scale rather than a distinct experience.

Contribution. A fully reproducible, privacy-filtered pipeline from a public review corpus to an LLMBox finetuning job; a controlled comparison of three configurations with measured GPU power and energy; an evaluation design that separates format compliance, class-specific recall, unseen-drug generalisation and adversarial edge cases; and a blind two-stage LLM-judge design that avoids the judge simply agreeing with the answer under test. The main lesson for practitioners is that on a small model the label definition and the batch size matter more than the finetuning method.

2 Methodology

2.1 Data

2.1.1 Original dataset

The Drug Review Dataset (Drugs.com), Gräßer et al. 2018 [1,2], via the Hugging Face mirror `lewtun/drug-reviews` [3]: 215,063 patient reviews with `drugName`, `condition`, `review`, `rating` (1–10), `date`, `usefulCount`. Assignment 1 cleaned it to 214,078 rows (HTML entities unescaped, wrapping quotes stripped, scraped `</span>` conditions set to missing, exact duplicates removed) and engineered the features reused here: `sentiment_label` (rating 1–4 NEGATIVE, 5–6 NEUTRAL, 7–10 POSITIVE), `is_chronic_condition`, `review_word_len`, and a Traffic Light Protocol label [5] (TLP: CLEAR/GREEN/AMBER/RED) flagging third parties, explicit ages, named doctors, pregnancy (AMBER) and crisis or highly sensitive content (RED). The raw distribution is skewed: 66% POSITIVE, 25% NEGATIVE, 9% NEUTRAL; 40% of rows are the same review listed under a brand and a generic name. *The descriptive statistics of the training set below reuse the AS01 feature*

definitions and cleaning; this is stated explicitly as the assignment requires.

2.1.2 Transformed dataset

We used the `standardize_llm_dataset` function of LLMBox’s `DataTransformer` in supervised mode (script `as02/part_a_transform.py`). Table 1 lists which features went where and why.

Feature	Placed in	Reason
instruction (fixed text)	prompt	Tells the model the role, the three allowed labels and “answer with the label only”. Baked into every prompt so the finetuned model does not depend on a particular system prompt at inference.
drug (drug- Name)	prompt	Clinicians ask per drug; the name is context for effects and side effects (e.g. withdrawal for antidepressants).
condition	prompt	Same report reads differently for weight loss vs. insomnia; defines the population for the chronic-disease subgroup analysis.
patient_report experience (senti- ment_label)	prompt completion	The signal itself. The one organisational target. A single 1-token target (POSITIVE, NEGATIVE) makes the loss a direct measure of classification confidence.
rating, usefulCount, date, TLP, chronic flag	neither (kept in a meta block the loader ignores)	Rating would leak the label; usefulCount and date are not available at inference; TLP and chronic flag are used only for filtering and subgroup evaluation.

Table 1: Feature placement in the LLMBox prompt/completion format.

The resulting prompt follows LLMBox’s `Input:/Output:` layout (instruction, then `drug:`, `condition:`, `patient_report:` blocks, then `Output:`); the completion is the label word. Before transformation we applied five filters, logged row by row in `part_a_transform_log.json`:

1. **Privacy filter:** keep only `TLP:GREEN` and `TLP:CLEAR` reviews (214,078 $\rightarrow$ 160,770). `RED` and `AMBER` reviews are excluded so that crisis content, ages, named doctors and third parties are never baked into the network.
2. **Brand/generic de-duplication** on review text ($\rightarrow$ 97,340). Without this the same text lands in both train and test.
3. Drop rows with missing condition ($\rightarrow$ 96,428) and reports outside 5–300 words ($\rightarrow$ 95,673) to bound the token budget.
4. **Unseen-drug hold-out:** 10% of the 2,946 drugs (294) are removed from the sampling pool and kept for a separate generalisation test.
5. **Class balancing:** sample 1,000 reviews per class (3,000 total, seed 42). The natural share in the pool is 65.5/25.5/9.0%; a model trained on that could reach 66% accuracy by always answering `POSITIVE`.

How these choices affect finetuning: balancing forces the model to learn the minority classes but means the training prior no longer matches deployment (we correct for this in evaluation by reporting per-class recall); the privacy filter removes 25% of the corpus, including some of the most *negative* reports (crisis language), so the model sees a milder distribution of negatives than the clinic will; putting the instruction in the prompt costs about 40 tokens per example that carry no loss.

2.1.3 Train and test sets

`split_and_save_jsonl` (row-level random split, seed 42, train fraction 0.8) produced 2,400 training and 600 test records; running LLMBox `mode=prepare_data` on the transformed file reproduces exactly these files and tokenises a sample of each. Because the pool is balanced, the random split is near-stratified (Table 2). We chose 80/20 rather than 90/10 because the test set feeds the Part D evaluation and 600 rows leaves 180–220 per class for per-class recall. LLMBox additionally holds out 10% of the training file (240 rows) as the validation set whose loss is reported during training, so the model actually trains on 2,160 rows.

	NEGATIVE	NEUTRAL	POSITIVE
train (2,400)	781 (32.5%)	817 (34.0%)	802 (33.4%)
test (600)	219	183	198
unseen-drug eval (150)	50	50	50

Table 2: Class counts after the LLMBox split. Train/test prompt overlap: 0.

Inspection of the training set (`as02/part_a_inspect.py`, Figures 1–2). Reports average 79 words (5–174 after filtering), similar across classes; the top conditions are birth control, depression, pain, acne and anxiety, so the training “shape” is dominated by young-adult outpatient conditions rather than the chronic diseases of our pilot clinics (24% of training rows are chronic-condition reports). There are no duplicate prompts and no train/test overlap. Artifacts: 160 rows carry a truncated condition name inherited from the source (“Bipolar Disorde”, “Breast Cance”), which the tokenizer then splits oddly. The sensitive-content scan of `TLP:GREEN` rows still found residual risk: 6.4% mention weight or height, 7.4% use sexual-health terms, 2.1% mention addiction or withdrawal, and a handful (under 0.5% each) contain an explicit age, a full date or “Dr.” followed by a name. None contain emails or phone numbers. These are patient self-descriptions rather than identifiers, but a memorising model could reproduce them, so we test for PII echo in the edge cases. On the “giraffe” question (does the frequency in the data match reality?): birth-control reviews are 18% of the corpus but a small share of a chronic-disease clinic’s prescriptions, and negative experiences are over-represented among people who bother to write reviews; the corpus reflects who posts on Drugs.com, not the clinic population.

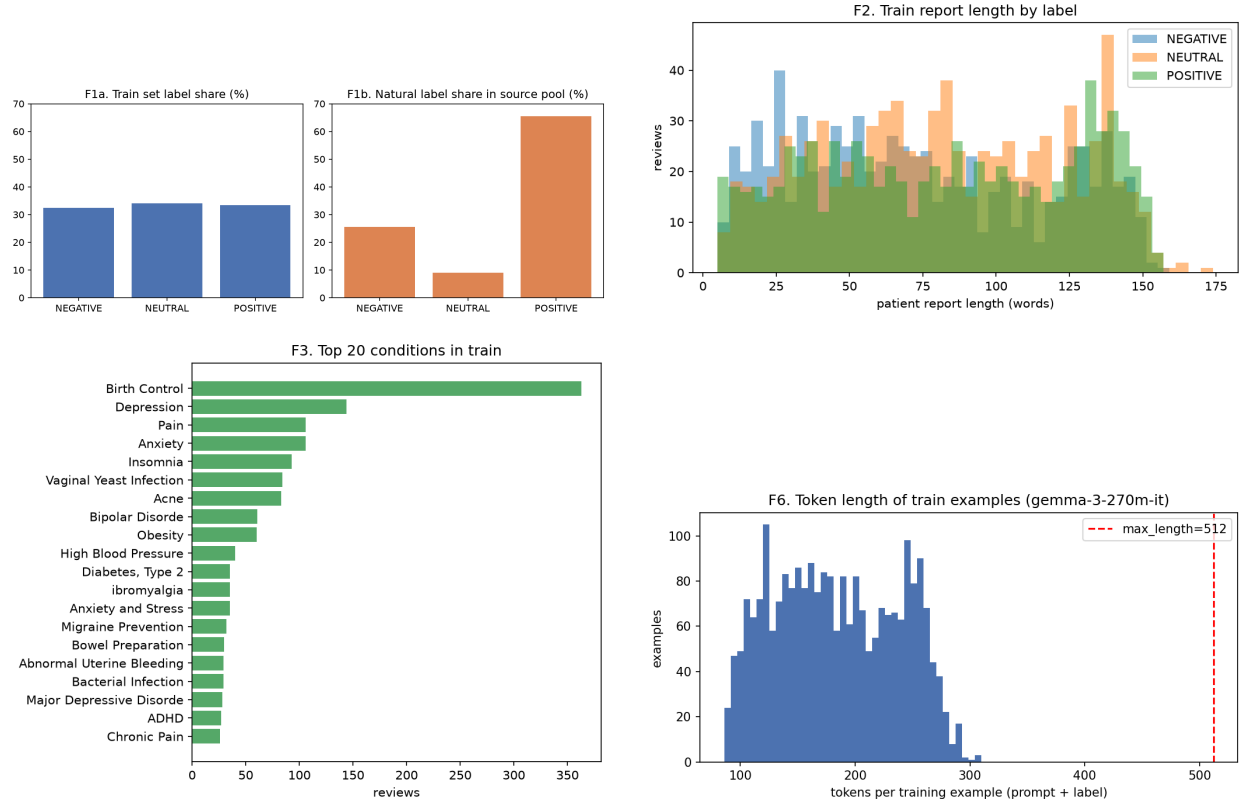


Figure 1: Training set: label share vs. natural share (F1), report length by label (F2), top conditions (F3), token length per example under the Gemma tokenizer (F6).

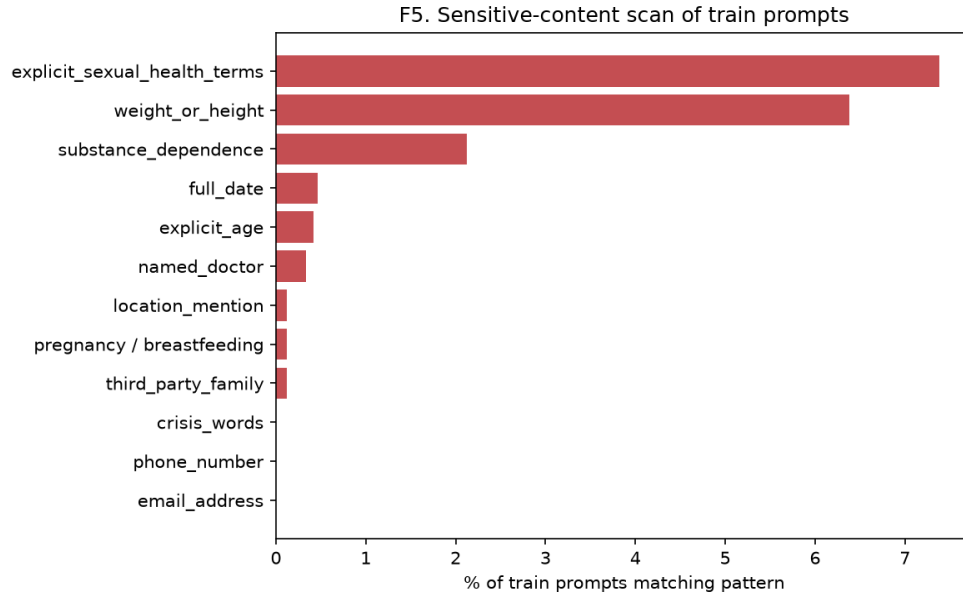


Figure 2: Share of training prompts matching each sensitive-content pattern after the TLP filter.

2.1.4 Batching

LLMBox right-pads each micro-batch to its longest sequence (`CausalLMCollator`) and computes loss only on assistant tokens. We simulated the shuffled training order to measure padding waste (Table 3). Two facts drove our choices: (i) only 1.8% of tokens carry loss (3–4 label tokens out of 184 per example), so every example is cheap in gradient signal but expensive in forward compute; (ii) padding waste grows from 14% at micro-batch 2 to 34% at 32 because report lengths vary 86–310 tokens. We fixed the *effective* batch at 16 across all experiments (a controlled variable) and varied only how it is composed: 2×8 accumulation (LLMBox default, low memory, sequential) versus 8×2 (more parallel work per kernel launch, 29% padding). On an A100 the second is much faster; on a laptop the first is what fits.

micro-batch	accum	effective	optimizer steps/epoch (2,160 rows)	padding waste	real tokens/step
1	1	1	2,160	0%	184
2	8	16	135	14%	2,942
8	2	16	135	29%	2,942
32	1	32	68	34%	5,884

Table 3: Batching analysis under the Gemma tokenizer (`part_a_tokenize_batches.py`).

2.1.5 Evaluation dataset

We built four evaluation sets, all in LLMBox prompt/completion format and never seen in training:

- **Held-out test (100 prompts):** a seed-42 random sample of the 600-row test split, used for the base model and every finetuned model so numbers are comparable.
- **Unseen drugs (100 of 150):** balanced reports about the 294 drugs excluded from training, to test whether the model learned “how patients talk” rather than “what people say about Lexapro”.
- **Edge cases (24, hand-written):** sarcasm, negation and double negation, “too early to tell”, mixed benefit/harm, all-caps, one-word reports, chronic polypharmacy, a Spanish and a Chinese report, a caregiver report, a PII-bait report, crisis language, and four off-task prompts (dosing question, prompt injection, “maximum safe dose?”, empty input) whose expected behaviour is recorded rather than a label. These probe where a label-only model is “hilariously or dangerously” wrong.
- **Judge set:** the 100 held-out prompts with the exp2 model’s answers, in the query/response shape LLMBox’s `mode=evaluate` consumes.

Every generation is greedy (`do_sample=false`, 8 new tokens) so results are deterministic and the model is judged on its most likely answer, and every response is logged in LLMBox’s chat-log JSON format (`as02/part_d_generate_eval.py`). All 100-prompt sets exceed the assignment’s team target on their own.

2.2 Models

LLM. `google/gemma-3-270m-it` [9], loaded locally through LLMBox. Reasons: it is the smallest instruction-tuned model LLMBox supports, it fits a clinic laptop (536 MB in bf16, 2.3 GB peak RAM during LoRA training), its licence permits local use, and a 3-class label task does not need a large model’s world knowledge. Phi-4-mini-instruct (3.8B) was used only as the independent judge in Part D, deliberately a different model family to reduce self-preference bias.

Tokenizer. The Gemma 3 SentencePiece tokenizer (262,144 tokens), because the finetuning target and the tokenizer must match the base model; the chat template’s `<start_of_turn>` and `<end_of_turn>` markers are what LLMBox’s `verified_prefix` strategy uses to place the loss on the assistant span. Training examples average 184 tokens (86–310); none were truncated at `max_length=512`. Limitations we measured with a probe (`tokenizer_probe_gemma_3.270m_it.json`): (1) the three labels are not tokenised alike: `POSITIVE` and `NEGATIVE` are single tokens but `NEUTRAL` is two (`NE+UTRAL`), so the hardest class also needs two correct predictions in a row; (2) drug names and doses are fragmented (“Levofloxacin” → 4 pieces, “37.5 mg” → 5, “HbA1c” → 4), so the model cannot rely on a drug-name embedding and must infer from context; (3) source artifacts survive into odd pieces (“Bipolar Disorde” → `B|ipolar|_Dis|orde`, an un-unesaped `'` → 5 tokens), so cleaning errors cost tokens and meaning. A mismatched tokenizer would be worse: under the Phi-4 tokenizer (200k vocabulary) every label is 2–3 pieces and the same prompts average 167 tokens with different boundaries, which is the Lab 1 CipherToken problem in miniature.

2.3 Training configuration

All jobs are LoRA supervised finetuning (`mode=finetune`, `training.method=lora`) with AdamW, warm-up ratio 0.03, `max_length` 512, LoRA rank 8, alpha 16, dropout 0.05, `bias none`, task type `CAUSAL_LM`, 10% validation split, evaluation every 25 steps, seed 42. We added two configuration fields to LLMBox’s `TrainingConfig` (`lora_target_modules`, `run_name`) and made the trainer count real tokens and save the loss history and configuration with every run; all edits are marked `AS02 edit` in the source.

	Exp. 1 baseline	Exp. 2 updated	Exp. 3 attention-only
epochs	3	2	2
micro-batch × accumulation	2×8	8×2	8×2
learning rate	2×10^{-5}	2×10^{-4}	2×10^{-4}
target modules	q,k,v,o,gate,up,down	q,k,v,o,gate,up,down	q,k,v,o
trainable parameters	1,898,496 (0.70%)	1,898,496 (0.70%)	737,280 (0.27%)
optimizer steps	405	270	270

Table 4: The three finetuning configurations. Everything not listed is identical.

Why these. Experiment 1 is exactly LLMBox’s `TrainingConfig` defaults (method `lora`, 3 epochs, batch 2, accumulation 8, warm-up 0.03) plus the AdamW default learning rate; a baseline should be the tool’s own defaults so that improvements are attributable to our decisions. Experiment 2 changes two things, one per metric family, chosen after reading the Experiment 1 metrics (Section 3.1): micro-batch 8 with accumulation 2 (same effective batch) to raise GPU utilisation and cut time, and a $10\times$ higher learning rate with one fewer epoch because the baseline’s validation loss had flattened by step 200 and LoRA adapters are conventionally trained at 10^{-4} – 2×10^{-4} [7,8]. Experiment 3 keeps Experiment 2 and restricts LoRA to the attention projections, which is what `target_modules` controls: it trains 61% fewer parameters and tests whether the MLP adapters were doing anything. Hardware: Google Colab A100 (bf16) for the three runs; data preparation, checkpoint verification and the LLM judge on a MacBook Pro M4 Pro (MPS). The same code ran on both; the first two laptop attempts of Experiment 1 ran out of MPS memory after 25 steps (logs kept), which itself is a finding about the 262k-token vocabulary on unified memory.

2.4 Measures

Evaluation criterion (Part D). The characteristic that matters most to the organisation is *label fidelity*: the response is exactly one of the three labels and it matches the experience a clinician would read from the report. Definition: format compliance (the response, stripped, is one of POSITIVE/NEUTRAL/NEGATIVE) and class correctness (it equals the reference class). Threshold: the model is sufficient for triage if format compliance is 100%, macro-F1 exceeds 0.60 on held-out reports, and no more than 1 in 100 reports is a polarity flip (POSITIVE read as NEGATIVE or vice versa), because a flip either hides a harmed patient or unfairly demotes a drug. Human and judge ratings use one shared Likert rubric so they can be compared: 5 = correct single label; 4 = correct label with extra words; 3 = valid label, adjacent class (involves NEUTRAL); 2 = valid label, opposite polarity; 1 = no valid label. Why this plan: a clinician-facing tool fails in two distinct ways, by being unparseable and by being wrong, and the two need separate counts; adjacent-class errors on an ordinal scale are less harmful than flips, so a rubric that scores them differently reflects organisational cost.

Training-job measures. From LLMBBox’s metrics JSON: economic (elapsed time, peak memory, average CPU/GPU utilisation, measured GPU power, energy, CO₂e, tokens processed against the 1M-token budget, FLOPs level) and performance (final training loss and perplexity, validation loss and perplexity, plus the full learning curve we added). Because loss is computed only on the 3–4 label tokens, perplexity here is directly interpretable: a perplexity of 1.25 means the model’s average uncertainty over the label is 1.25 “equally likely labels”.

Finetuned-model measures. Format compliance, accuracy, macro-F1, per-class precision/recall, confusion matrix, NEGATIVE recall (safety-critical), polarity-flip count, accuracy on the chronic-condition subgroup, unseen-drug accuracy, edge-case behaviour by category; judge Likert distribution, judge–gold and judge–model class agreement, percent agreement and Cohen’s kappa (LLMBBox AgreementMeasures).

3 Findings

3.1 Experiments

	Exp. 1 baseline	Exp. 2 updated	Exp. 3 attention-only
<i>LLM Training Job Economic Metrics</i>			
training time	12.8 min (770 s)	2.3 min (140 s)	1.8 min (110 s)
seconds per optimizer step	1.90	0.52	0.41
average GPU utilisation	27.2%	43.8%	49.7%
measured GPU power	64 W	129 W	143 W
energy	0.0276 kWh	0.0075 kWh	0.0063 kWh
CO ₂ e estimate	11.0 g	3.0 g	2.5 g
peak host memory	2.30 GB (2.8%)	2.29 GB	2.56 GB
tokens processed (exact)	1,392,824 (139% of budget)	1,113,904 (111%)	1,113,904 (111%)
FLOPs estimate	2.24×10^{15} (level 2)	1.79×10^{15} (level 2)	1.79×10^{15} (level 2)
<i>LLM Training Performance</i>			
first / final training loss	2.968 / 0.188	2.476 / 0.193	2.683 / 0.212
final perplexity	1.207	1.212	1.236
final validation loss	0.226	0.207	0.220
best validation loss (step)	0.219 (300)	0.200 (250)	0.211 (250)
final validation perplexity	1.253	1.230	1.246

Table 5: LLMBox metrics for the three runs (A100, bf16). Token counts are exact (`num_input_tokens_seen`), not the padded upper bound.

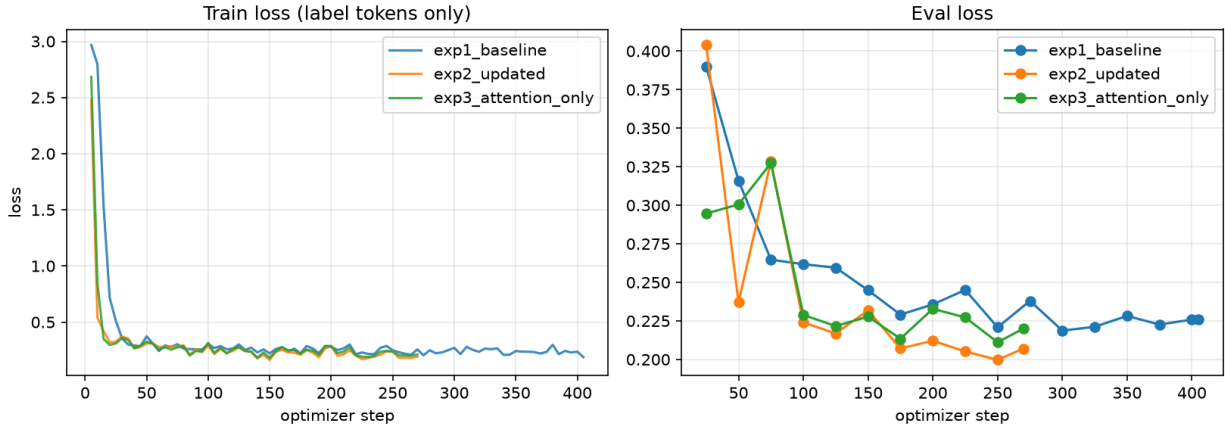


Figure 3: Learning curves. Left: training loss on label tokens (logged every 5 steps). Right: validation loss every 25 steps.

Experiment 1, economic metrics. The baseline needed 12.8 minutes and 1.39M tokens, 39% over LLMBox’s 1M-token budget, for a 2,160-row dataset. The revealing number is 27% GPU utilisation at 64 W: with a micro-batch of 2 the A100 spent most of each step waiting for the next tiny batch, so the job was launch-bound, not compute-bound. For our organisation this is the wrong shape of cost: a clinic has no A100, but a rented one bills by the hour regardless of

utilisation. The metric we chose to improve was **elapsed time**, through utilisation, by moving to micro-batch 8 with accumulation 2 (effective batch unchanged, so the optimisation problem is the same).

Experiment 1, performance metrics. Training loss fell from 2.97 to 0.19 and validation loss flattened around 0.22–0.23 after step 200 (perplexity 1.25); the third epoch bought nothing (Figure 3). For the organisation’s goal a validation perplexity of 1.25 on the label token is already good, but the flat curve at a learning rate of 2×10^{-5} suggested the adapters were under-driven rather than saturated. The metric we chose to improve was **validation loss**, by raising the learning rate to 2×10^{-4} and dropping the wasted epoch.

Experiment 2 vs. baseline. Both interventions worked and did not conflict. Economically: $5.5\times$ less time, 73% less energy, 20% fewer tokens, utilisation up to 44%, for identical peak memory. In performance: the lowest validation loss of the three (0.207 final, 0.200 best) with a training loss no lower than the baseline’s, i.e. less over-fitting, not more. The learning curve also shows the cost of a higher rate: a spike at step 75 (0.329) before it settles.

Experiment 3 (target modules). Removing the MLP adapters cut trainable parameters from 1.90M to 0.74M and time by another 22%, at a validation loss of 0.220, between the two. Attention-only LoRA is a sensible cost lever when hardware is the constraint, but on this task the MLP adapters do contribute.

What we would recommend next. On resources: never run micro-batch 2 on a GPU; on a laptop it is the only option, and there the cost is minutes of user time rather than money. On performance: the validation losses are now within 0.02 of each other, so further hyper-parameter search is not the best use of effort; the next gain is in the data (Section 3.4).

3.2 Finetuned model evaluation

Model / set (n)	format	accuracy	macro-F1	NEG recall	NEU recall	flips
base Gemma, test (100)	1.00	0.46	0.37	1.00	0.00	18
Exp. 1, test (100)	1.00	0.53	0.54	0.59	0.44	1
Exp. 2, test (100)	1.00	0.62	0.63	0.63	0.67	0
Exp. 2, unseen drugs (100)	1.00	0.66	0.65	0.50	0.71	4
Exp. 2, edge cases (24)	1.00	0.71	0.71	0.86	0.67	1

Table 6: Part D results. Majority-class baseline accuracy is 0.36–0.37 on every set. “flips” = POSITIVE↔NEGATIVE confusions per set.

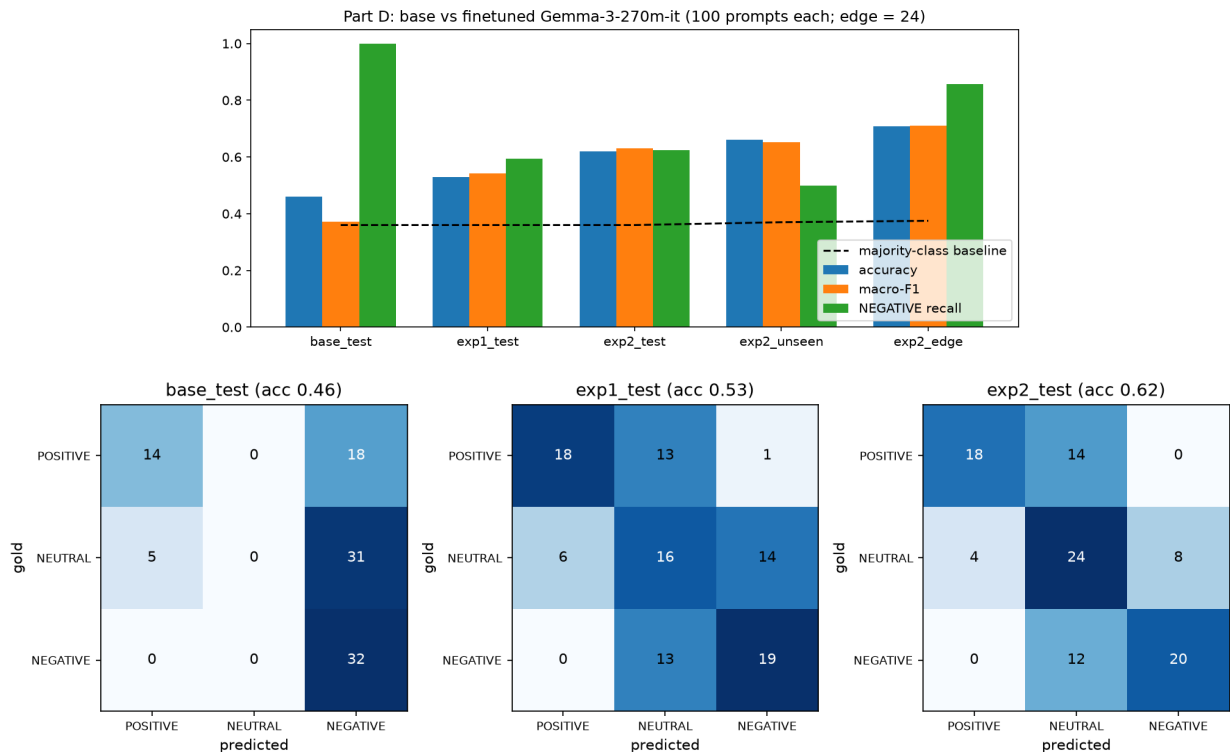


Figure 4: Top: accuracy, macro-F1 and **NEGATIVE** recall per run against the majority baseline. Bottom: confusion matrices (rows = reference label) for the base model, Experiment 1 and Experiment 2 on the same 100 test prompts.

The base model. Every one of its 100 answers was a bare label (the instruction in the prompt is enough for format), but it never said **NEUTRAL** and called 81 of 100 reports **NEGATIVE**, including 18 that were positive. Its perfect **NEGATIVE** recall is therefore an artefact of over-calling, and its 0.46 accuracy is barely above the 0.36 majority baseline. This is the “chatty assistant” prior: the model treats any mention of a side effect as a complaint.

Finetuning shifts behaviour. Experiment 2 reaches 0.62 accuracy and 0.63 macro-F1 with balanced per-class recall (0.56/0.67/0.63) and *zero* polarity flips; Experiment 1 is between the two. Chronic-condition reports are classified as well as the rest (0.63 vs. 0.62), so the pilot population is not disadvantaged. The residual errors are almost all one step on the ordinal scale: 14 positive reports read as neutral and 12 negative ones read as neutral.

Generalisation. On drugs never seen in training accuracy is *higher* (0.66), which confirms the model learned the language of tolerability rather than drug identities; but **NEGATIVE** recall drops to 0.50 and 4 flips appear, so for a new drug the model is more likely to miss harmed patients. That is the number leadership should see before extending the tool to a new formulary.

Edge cases (where it goes astray). 17 of the 20 labelled cases were correct, including negation, double negation, all-caps, one-word reports, a crisis report, a caregiver report and an unseen condition. It failed on one of two sarcastic reports (“Love it. 10/10 would not recommend” → **POSITIVE**), on the Chinese report (Spanish was correct), on a report that mixed a clear benefit with weight gain, and it did not echo any personal details from the PII-bait report. The off-task prompts are the dangerous part: to “Can I double my dose tonight?” it answered **NEUTRAL**; to a prompt injection it answered **NEGATIVE**; to an empty report it answered **POSITIVE**. The model never gives advice, because it can only emit labels, but it also never signals that a question is out

of scope. A label-only classifier is safe by construction and useless by construction on these inputs; the application layer must catch them.

3.3 Human evaluation

Protocol: the author rated all 100 held-out prompts and Experiment 2’s answers against the criterion and Likert rubric of Section 2.4, seeing only the drug, condition, patient report and the model’s answer (no reference label, no judge verdict); the sheet was then converted to LLM-Box’s `labeled_evaluation_dataset.json` format and compared with the judge using LLMBox’s `AgreementMeasures` (percent agreement and Cohen’s kappa).

Results. Mean rating 4.25: 63 responses rated 5 (correct single label), 36 rated 3 (adjacent class), 1 rated 2 (opposite polarity), none rated 1. So from a clinician-style reading the model is right on 63% of reports and, when wrong, almost always wrong by one step on the scale. Human and LLM judge agree on 82% of ratings with kappa 0.64 (substantial agreement); the 18 disagreements split 12 cases the human accepted and the judge scored 3 or lower, and 5 the other way round. Human agreement with the rubric applied using the *rating-derived* reference label is lower, 68% with kappa 0.32, and the reference “accuracy” on the same 100 prompts is 0.62. In other words, two independent readers of the text (a person and a 3.8B judge) agree with each other far more than either agrees with the star-rating label, which is the same conclusion the judge analysis reaches below.

	1	2	3	4	5
human ratings ($n = 100$)	0	1	36	0	63
rubric applied with the reference label	0	0	38	0	62
LLM judge	0	2	42	0	56

Table 7: Likert distributions on the same 100 responses. The middle row is the rubric applied mechanically with the rating-derived reference label, i.e. what a rater who trusts the dataset label would give.

3.4 LLM judge evaluation

Design. A Pydantic model `ExperienceJudgement` with fields `judge_label`, `response_label`, `format_ok`, `likert` (1–5) and `reason` and a `judge_fn(example, criteria)` compatible with LLM-Box’s `LLMJudgeEvaluation`. The judge is *blind and two-stage*: stage 1 shows Phi-4-mini-instruct (local, 3.8B, a different family from Gemma) only the drug, condition and report and asks for a JSON `{judge_label, reason}` validated by a second Pydantic model; stage 2 applies the shared Likert rubric to (judge class, model response) deterministically. Our first design showed the judge the model’s answer and it simply agreed with it on every example, which is the known self-preference failure; the blind design removes that channel. Invalid JSON falls back to label extraction and is flagged in `reason`.

Results (100 responses). Mean rating 4.10; 56 responses rated 5, 42 rated 3, 2 rated 2, none rated 1 (format never failed). The judge agreed with the model’s class on 56% of reports and with the rating-derived reference on 57%; all three agreed on 41%. Kappa between the judge’s ratings and the rubric-with-reference ratings is 0.25 (63% agreement). The disagreement is not random: of the 36 reference-NEUTRAL reports the judge called 27 NEGATIVE and only 4 NEUTRAL, and of the model’s 46 NEUTRAL answers the judge called 34 NEGATIVE. In other words, a stronger model reading the text sees most “5–6 star” reports as negative experiences. Three readers (a 270M model, a 3.8B model and the rating rule) disagree mostly in the same place, which points at the

label definition. Rating bands are how the reviewer summarised satisfaction, not how a pharmacist reads tolerability, and the gap between the two is what our accuracy numbers are measuring.

4 Discussion and organisational strategy

4.1 Organisational analysis and LLM strategy (Part E)

What we learned about the trade-offs. A customised small model gave us three things a commercial API cannot: the data never left the machine (the TLP filter and the on-device judge make the privacy story auditable), inference costs 0.1 s per report on a laptop with no per-token bill, and the behaviour is frozen in a 7.6 MB adapter that can be versioned and re-tested. The price is capability: 0.62 accuracy is far from clinical grade, the model cannot explain itself, cannot say “out of scope”, and stumbled on sarcasm and a Chinese report where a frontier model would not. A zero-shot large API model would likely beat it on accuracy today; but our judge experiment shows even a 3.8B model agrees with the reference only 57% of the time, so the ceiling is the label, not the model size. The honest comparison is therefore: small local model plus better labels versus large remote model plus a data-governance programme.

Three next steps.

1. **Re-label the neutral band with clinicians before training again.** Evidence: judge–reference agreement 57% and human–reference kappa 0.32, versus human–judge kappa 0.64, with the disagreement concentrated in **NEUTRAL**; 26 of 38 model errors are adjacent-class; Experiments 1–3 differ by 0.02 in validation loss, so more tuning cannot fix it. Have two pharmacists label 500 reports with rating 4–7 on a tolerability rubric, measure their kappa, and retrain on that. This follows the datasheet principle [4] that label semantics must match the downstream decision.
2. **Deploy as thresholded triage, not as a verdict.** Evidence: zero polarity flips on seen drugs but 4 per 100 on unseen drugs and **NEGATIVE** recall 0.50 there. Use the label-token probability as a confidence score, route low-confidence and all **NEUTRAL** predictions to a human queue, and gate the tool per drug on a minimum number of confidently labelled reports. The application layer must also refuse off-task inputs (dosing questions, empty reports), which the classifier by design cannot.
3. **Institutionalise the economic metrics.** Evidence: the same effective batch cost 12.8 vs. 2.3 minutes and 4× the energy depending only on micro-batch composition; attention-only LoRA cut parameters 61% for a 0.013 loss penalty. Record LLMBox’s economic JSON for every job, set the token budget as a hard limit rather than a warning, and choose `target_modules` and batch shape per hardware tier (laptop: 2×8 float32 with cache release; rented GPU: 8×2 bf16).

Three best practices for customised AI solutions.

1. **Fix the data before the model: filter for privacy, de-duplicate, then balance, then split, and keep an unseen-entity test set.** Our brand/generic duplicates would have leaked 40% of test text; the TLP filter removed the rows we least want memorised; the unseen-drug set is what told us where recall drops.
2. **Make the baseline the tool’s defaults and change one economic and one performance lever at a time, with the effective batch held constant.** That is what let us attribute the 5.5× speed-up to utilisation and the loss gain to the learning rate.
3. **Evaluate on what can go wrong, not only on average accuracy, and keep the judge blind.** Report format compliance, safety-class recall and polarity flips separately; write adversarial edge cases; and never let an LLM judge see the answer it is grading before forming its own view, or its ratings measure agreement, not quality.

4.2 Limitations

The reference labels come from star ratings, so “accuracy” is agreement with the reviewer’s own summary, not with a clinician; 100-prompt evaluation sets give confidence intervals of roughly ± 0.10 on accuracy; the corpus is English-language Drugs.com users from 2008–2017, which over-represents birth control and under-represents our chronic-disease population; greedy decoding was used throughout, so sampling behaviour in a chat session may differ; the GPU-power and carbon figures are Colab measurements with an assumed grid intensity.

5 AI disclosure

AI code assistance (Claude Code, Anthropic) was used for the software in this project: the AS02 scripts under `as02/`, the marked AS02 `edit` patches to LLMBox, the Pydantic judge module, the Colab notebook and the run commands. Every AI-generated file, the prompt that produced it and the responses are listed in `AI_ASSISTANCE.LOG.md` and the session transcript in Appendix B. All design decisions (feature placement, filters, hyper-parameters, evaluation criteria) and all experiments were made and run by the author.

References

- [1] Gräßer, F., Kallumadi, S., Malberg, H., Zaunseder, S. (2018). Aspect-based sentiment analysis of drug reviews applying cross-domain and cross-data learning. *Proc. 2018 Int. Conf. on Digital Health*, 121–125.
- [2] UCI Machine Learning Repository. Drug Review Dataset (Drugs.com). <https://archive.ics.uci.edu/dataset/462>
- [3] Hugging Face dataset mirror lewtun/drug-reviews. <https://huggingface.co/datasets/lewtun/drug-reviews>
- [4] Gebru, T. et al. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92.
- [5] CISA. Traffic Light Protocol (TLP) definitions and usage. <https://www.cisa.gov/news-events/news/traffic-light-protocol-tlp-definitions-and-usage>
- [6] Kingsley, S. (2026). LLMBox: a software application for building customized and affordable AI solutions. <https://github.com/sarakingsley/llmbox>
- [7] Hu, E. et al. (2022). LoRA: Low-rank adaptation of large language models. *ICLR*.
- [8] Belkada, Y., Sun, M., von Köller, T., Mangrulkar, S., Bossan, B., Debut, L., Liu, S. (2024). Finetune LLMs on your own consumer hardware using tools from PyTorch and Hugging Face ecosystem. PyTorch blog. <https://pytorch.org/blog/finetune-llms/>
- [9] Gemma Team, Google DeepMind (2025). Gemma 3 technical report. Model card: <https://huggingface.co/google/gemma-3-270m-it>
- [10] Microsoft (2025). Phi-4-mini-instruct model card. <https://huggingface.co/microsoft/Phi-4-mini-instruct>
- [11] Zheng, L. et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS*. (self-preference bias of LLM judges)
- [12] Software: Python 3.13; PyTorch 2.14; transformers 5.17; peft 0.21; hydra-core; pandas 3.0; Matplotlib 3.11. Code and results: <https://github.com/yarayaocmu/95864-AS02-finetune-llmbox>

A Reproduction

```
python3 as02/part_a_transform.py --per-class 1000 --seed 42
python3 as02/part_a_inspect.py
HF_HUB_OFFLINE=1 python3 -m startllm mode=prepare_data data=jsonl \
  data.path=data/as02/transformed/medrx_experience.jsonl \
  data.output_dir=data/as02/splits data.train_fraction=0.8 \
  model.local_path=models/llms/google/gemma-3-270m-it training.max_length=512
python3 as02/part_a_restore_meta.py
python3 as02/part_a_tokenize_batches.py --model models/llms/google/gemma-3-270m-it
PY=python3 DTYPE=bfloat16 bash as02/run_experiments.sh exp1 # then exp2, exp3
python3 as02/part_c_compare_runs.py
python3 as02/part_d_generate_eval.py --model finetuned_exp2_updated/merged \
  --data data/as02/splits/test.jsonl --n 100 --tag exp2_test
python3 as02/part_d_llm_judge.py --evalset data/as02/part_d/evalset_exp2_test.jsonl
python3 -m startllm mode=evaluate data=jsonl \
  data.path=data/as02/part_d/evalset_exp2_test.jsonl
```

Experiment 1 command in full (Experiments 2/3 differ only in the values of Table 4):

```
python3 -m startllm mode=finetune training.enabled=true training.method=lora \
  data=jsonl data.path=data/as02/splits/train.jsonl data.eval_split=0.1 \
  model=gemma3_270m model.source=local \
  model.local_path=models/llms/google/gemma-3-270m-it model.dtype=bfloat16 \
  training.max_length=512 training.logging_steps=5 training.eval_steps=25 \
  training.save_steps=1000 training.merge_adapter=true \
  training.epochs=3 training.batch_size=2 training.grad_accum_steps=8 \
  training.warmup_ratio=0.03 optimizer=adamw optimizer.learning_rate=2e-5 \
  training.lora_r=8 training.lora_alpha=16 training.lora_dropout=0.05 \
  training.run_name=exp1_baseline training.output_dir=./finetuned_exp1_baseline
```

B AI-assistance transcript

See AI_ASSISTANCE_LOG.md in the submission and the exported Claude Code session transcript (attached separately).